Radova "Latest" Page Plan: Issue List

The Latest page features Radova's newest updates across 4 categories: **Agenda, Article, News, Security**.

- **Tech Stack:** Pure static HTML, HTMX, Pico CSS, and Vanilla JavaScript (zero Node.js/npm dependencies, fully readable in Lynx).
- **Core Focus:** Development (HTML Structure & Logic), Styling (UI/UX & Accessibility), and Copywriting (UI Text & Content).
- **Issue Format:** Goal, Requirement, Procedure, Result.

How to Use This File

1. Create the **Milestones** (GitHub: *Issues* → *Milestones*) and **Labels** (*Issues* → *Labels*) based on the lists below.
2. Create each issue in order. Copy and paste the title and body, then assign the corresponding Milestone and Labels.
3. The numbers (1, 2, 3...) indicate the execution order, not the actual GitHub issue numbers.

Milestones

- **Phase 1: Foundation & Setup**
- **Phase 2: Styling & UI Layout**
- **Phase 3: Interactive Features & Feeds**
- **Phase 4: Copywriting & Launch Content**

Labels

- **Focus Area:** `focus:develop` , `focus:styling` , `focus:copywriting`
- **Content Type:** `type:agenda` , `type:article` , `type:news` , `type:security`
- **Priority:** `priority:high` , `priority:medium` , `priority:low`
- **Size:** `size:small` , `size:medium` , `size:large`

PHASE 1: FOUNDATION & SETUP

Issue 1

Title: Set up directory structure and content metadata standards
Milestone: Phase 1: Foundation & Setup
Labels: `focus:develop` , `priority:high` , `size:small`

Goal

- [ ] Establish a clean directory structure and standardized metadata fields across all four content types without any external runtime bloat.

Requirement

- [ ] Agreed URL structure: `/latest/` , `/latest/<type>/` , `/latest/<type>/<slug>/` , and `/latest/<type>/page/<number>/` .

Procedure

- [ ] Create core directories: `latest/` (with subfolders: `agenda`, `article`, `news`, `security`), `templates/` (reusable HTML partials/shells), and `assets/` (CSS, JS, fonts).
- [ ] Define standard metadata fields for all posts: `title`, `slug`, `date`, `summary`, `tags`.
- [ ] Define additional **Agenda** fields: `start_time`, `end_time` (optional, WIB/+07:00 timezone), `location`, `register_url` (optional), `status` (`scheduled` or `cancelled`).
- [ ] Define additional **Security** fields: `advisory_id` (e.g., `RSA-2026-001`), `severity` (`low`, `medium`, `high`, `critical`), `affected_versions`, `patched_versions`, `cve` (optional), `status` (`investigating`, `mitigated`, `resolved`, `informational`).
- [ ] Create 1 sample placeholder entry in each category folder for layout and styling tests.

Result

- [ ] Directory tree matches the target URL paths cleanly.
- [ ] All four sample entries are in place with consistent metadata structures.

Issue 2

Title: Build core HTML page templates and listing structure
Milestone: Phase 1: Foundation & Setup
Labels: `focus:develop`, `priority:high`, `size:medium`

Goal

- [ ] Create clean, semantic static HTML structures for index pages, category listings, paginated views, and detail pages.

Requirement

- [ ] Issue 1 completed.

Procedure

- [ ] Build the main `/latest/index.html` listing (displaying the 10 newest items overall), category index pages (`/latest/<type>/index.html`), and paginated pages (10 items per page).
- [ ] Structure the **Agenda** index into two distinct semantic sections: "Upcoming" (sorted by closest date) and "Past Events" (sorted by newest) using WIB (`+07:00`) timestamps.
- [ ] Include reading time metadata on **Article** pages.
- [ ] Ensure all HTML is strictly semantic and free of inline scripts or styles.

Result

- [ ] All main index, category, and detail HTML pages can be served directly by any static web server.
- [ ] The Agenda page clearly separates upcoming and past events.

PHASE 2: STYLING & UI LAYOUT

Issue 3

Title: Build base styling, brand theme, and master layout
Milestone: Phase 2: Styling & UI Layout
Labels: `focus:styling`, `priority:high`, `size:medium`

Goal

- [ ] Align the Latest page's visual identity with the Radova homepage (colors, typography, header, footer) while keeping it fully readable in Lynx.

Requirement

- [ ] Issue 1 completed.
- [ ] Local font files (Lato & Roboto), Pico CSS, and HTMX stored in `assets/`.

Procedure

- [ ] Build the master HTML layout: "Skip to main content" link, `<header>`, category navigation tabs, `<main id="main">`, and `<footer>`.
- [ ] Override `--pico-*` CSS variables in `assets/site.css` using the Radova color palette: Navy `#0E0347`, Reds `#A10000`, `#BB080B`, `#DB000F`, and Lavender `#F7F6FF` (without using `!important`).
- [ ] Apply Lato for headings and Roboto for body text.
- [ ] Add secure HTMX configuration in `<head>`: `<meta name="htmx-config" content='{"allowEval":false,"allowScriptTags":false,"selfRequestsOnly":true,"includeIndicatorStyles":false}'>`.
- [ ] Structure semantic HTML elements so the reading order flows logically from top to bottom in Lynx (`lynx -dump -width=80`).

Result

- [ ] Zero inline CSS or JavaScript inside the HTML files.
- [ ] No horizontal scrolling across 375px (mobile), 768px (tablet), and 1280px (desktop) viewports.
- [ ] The base layout reads cleanly and sequentially in Lynx.

Issue 4

Title: Style content list pages and card components
Milestone: Phase 2: Styling & UI Layout
Labels: `focus:styling`, `priority:high`, `size:medium`

Goal

- [ ] Present clean, scannable content lists on `/latest/` and category pages that do not rely solely on color to convey meaning.

Requirement

- [ ] Issue 2 and Issue 3 completed.

Procedure

- [ ] Style list items to display: title (link), content type badge, publication date (`<time datetime>`), and short summary.
- [ ] Create distinct visual metadata for **Security** items (display advisory ID & severity badge) and **Agenda** items (display event date/time, location, & a "Cancelled" badge if applicable).
- [ ] Ensure severity and status badges include explicit text labels (e.g., `[Severity: High]`), not just colors or icons.
- [ ] Apply CSS word-wrapping so long titles wrap cleanly to the next line without overflowing their container.
- [ ] Style the pagination controls (Previous / Next) at the bottom of the list.

Result

- [ ] All text and badge color contrasts meet WCAG AA accessibility standards.
- [ ] In Lynx, every list item displays its status/severity as plain text, and listing pages contain no more than 30 links total.

Issue 5

Title: Style detail pages for all 4 content types
Milestone: Phase 2: Styling & UI Layout
Labels: `focus:styling` , `type:agenda` , `type:security` , `priority:high` , `size:medium`

Goal

- [ ] Provide a comfortable reading layout for each content type while highlighting category-specific metadata.

Requirement

- [ ] Issue 2 and Issue 3 completed.

Procedure

- [ ] Style **Article & News** typography: paragraph spacing, blockquotes, horizontally scrollable code blocks for small screens, metadata bar (date, author, reading time, tags), and Previous/Next post navigation.
- [ ] Style the **Agenda** detail box at the top of the page using a semantic description list (`<dl>`) showing start/end times (WIB), location/online link, event status, and registration CTA button.
- [ ] Style the **Security** metadata header using `<dl>` to display the advisory ID, severity level, status, affected versions, patched versions, and CVE links.
- [ ] Format standard Security Advisory body sections: Summary, Impact, Mitigation, Credits, and Timeline.

Result

- [ ] Wide code blocks or tables scroll internally without breaking the mobile page width.
- [ ] Agenda and Security metadata use `<dl>` , `<dt>` , and `<dd>` tags, rendering cleanly in Lynx.

PHASE 3: INTERACTIVE FEATURES & FEEDS

Issue 6

Title: Implement fast tab switching and "Load More" pagination (HTMX)
Milestone: Phase 3: Interactive Features & Feeds
Labels: `focus:develop` , `priority:high` , `size:medium`

Goal

- [ ] Enable instant category tab switching and "Load More" pagination without full page reloads, while keeping standard link navigation when JavaScript is disabled.

Requirement

- [ ] Issue 4 completed.

Procedure

- [ ] Add HTMX attributes to category tabs (`hx-boost="true"` , `hx-select="main"` , `hx-target="main"` , `hx-swap="outerHTML"` , `hx-push-url="true"`) and mark the active tab with `aria-current="page"` .
- [ ] Enhance the "Next Page" link with HTMX to append the next 10 items directly to the current list, hiding the button when the final page is reached.
- [ ] Add a lightweight script in `assets/site.js` to manage keyboard focus after HTMX swaps and announce newly loaded items to screen readers via `aria-live="polite"` .

- [ ] Disable HTMX (`hx-boost="false"`) on external links, downloadable files, and XML feeds.

Result

- [ ] With JavaScript enabled, switching tabs and loading more items works seamlessly, and browser Back/Forward buttons update the URL properly.
- [ ] With JavaScript disabled (or in Lynx), tabs and pagination links work as standard HTML anchor links.

PHASE 4: COPYWRITING & LAUNCH CONTENT

Issue 9

Title: Write UI microcopy, empty states, and 404 page copy
Milestone: Phase 4: Copywriting & Launch Content
Labels: `focus:copywriting` , `focus:styling` , `priority:high` , `size:small`

Goal

- [ ] Ensure all interface microcopy, empty category states, and the 404 error page have clear, helpful, and consistent wording.

Requirement

- [ ] Issue 4 and Issue 5 completed.

Procedure

- [ ] Write short introductory descriptions (1–2 sentences) for the header of `/latest/` and each category page (`Agenda` , `Article` , `News` , `Security`).
- [ ] Write clear empty-state copy for categories with no published entries yet (e.g., for Security: *"No security advisories have been published yet."* with a link back to `/latest/`).
- [ ] Write copy and build the `404.html` page with a helpful message and a direct link back to `/latest/` .
- [ ] Standardize all button and navigation labels (e.g., "Skip to main content", "Load more", "Add to calendar (.ics)").

Result

- [ ] Empty categories display a styled, informative notice rather than a blank or broken layout.
- [ ] The 404 page matches the Radova theme and reads cleanly in Lynx.

Issue 10

Title: Write Security Policy and vulnerability reporting copy
Milestone: Phase 4: Copywriting & Launch Content
Labels: `focus:copywriting` , `type:security` , `priority:high` , `size:small`

Goal

- [ ] Provide clear instructions for security researchers and users on how to privately report vulnerabilities to Radova.

Requirement

- [ ] Official security contact email finalized.

Procedure

- [ ] Write a concise Security Policy (for `SECURITY.md` and linked prominently on `/latest/security/`) covering: private reporting steps, scope, expected response timeline, and coordinated disclosure policy.
- [ ] Write a brief explainer on `/latest/security/` detailing Radova's severity ratings (Low, Medium, High, Critical) and advisory ID format (`RSA-YYYY-NNN`).
- [ ] Create `/.well-known/security.txt` containing the contact address, policy URL, preferred languages, and expiration date.

Result

- [ ] Visitors on the Security page can immediately find instructions for reporting vulnerabilities privately.
- [ ] The copy is straightforward and `/.well-known/security.txt` is valid and accessible.

Issue 11

Title: Write initial content for Soft Launch
Milestone: Phase 4: Copywriting & Launch Content
Labels: `focus:copywriting`, `type:article`, `type:news`, `type:agenda`, `priority:high`, `size:medium`

Goal

- [ ] Populate the Latest section with real, accurate, and engaging initial content for the public soft launch.

Requirement

- [ ] Issue 2, Issue 5, and Issue 9 completed.

Procedure

- [ ] **News:** Write 1 launch announcement introducing the new Radova Latest page and what visitors can expect from each category.
- [ ] **Article:** Write 1 introductory technical article about Radova Linux's core philosophy and architecture (stick strictly to verifiable facts; avoid unverified claims).
- [ ] **Agenda:** Write 1 upcoming community event or discussion session (if no real event is scheduled yet, remove sample placeholders so the empty state from Issue 9 displays).
- [ ] **Security:** Remove the sample security placeholder from `latest/security/` so the category launches cleanly with its empty state.
- [ ] Proofread all copy and perform a final visual check across desktop, mobile, and Lynx.

Result

- [ ] All sample test files are removed and replaced with production-ready launch content.
- [ ] All pages render cleanly without errors and are ready for deployment.